

Trigger BANK V2: completar bancos del CSV sin duplicar

****Carpeta:**** `/Users/kiberno/Documents/Requerimientos/TriggerBank/V2`

****Motor:**** PostgreSQL

****Fecha:**** 1 de septiembre de 2026

****Versión:**** V2 (no sustituye los archivos de V1 en `TriggerBank/`; es entrega independiente)

V1 solo sembraba empresas que ****ya tenían**** filas en `bank`. V2 recorre ****`public.enterprise`**** (activas) y cubre empresas sin bancos.

Objetivo

Tras cada `INSERT` en `public.bank`, completar los bancos venezolanos del catálogo (`banks\_venezuela.csv`, 25 filas) que ****aún no existan**** en esa empresa.

Lo que ya está en `bank` ****prevalece siempre****: ****nunca hay `UPDATE`****. El CSV solo inserta faltantes.

Precedencia (BANK gana)

Una fila del CSV ****no se inserta**** si, en la misma `id\_enterprise`, ya hay un banco que coincida por ****código o nombre**** (normalizados). Las filas con `co\_operation = 'D'` también cuentan como existentes (no se reinserta un borrado lógico).

Caso	En BANK	En CSV	Resultado
---	---	---	---
Falta el banco	-	0102 / BANCO DE VENEZUELA	INSERT
Mismo código, otro nombre	0102 / "Mi banco"	0102 / BANCO DE VENEZUELA	No inserta ni pisa. Queda el nombre de BANK
Nombre equivalente	"Venezuela" o "Banco de Venezuela"	BANCO DE VENEZUELA	No inserta (duplicado de nombre)
Código ocupado por otro banco	BANESCO / 0102	0102 Venezuela y 0134 Banesco	No entra Venezuela (0102 ocupado). No entra segundo Banesco (nombre)

Normalización

- ****Código:**** solo dígitos, `lpad` a 4 (`102` = `0102`).
- ****Nombre:**** mayúsculas, sin acentos, sin `C.A.` / `S.A.`, se quitan sufijos finales `BANCO UNIVERSAL`, `BANCO MICROFINANCIERO`, `BANCO DIGITAL`.
- ****Subcadena:**** si un nombre contiene al otro (mínimo 6 caracteres), p.ej. `VENEZUELA` dentro de `BANCO DE VENEZUELA`.
- ****Falsos positivos:**** "BANCO" (5 letras) no matchea. Un nombre suelto "CREDITO" (7) podría chocar con Venezolano de Crédito y Nacional de Crédito; es raro como fila real.

Ámbito: ****por empresa**** (`id\_enterprise`). El catálogo no es global.

Archivos

Archivo	Uso
---	---
`banks_venezuela.csv`	Catálogo fuente (25 bancos)
`trg_bank_seed_ve.sql`	Install: catálogo, funciones, trigger, backfill
`trg_bank_seed_ve_run.sql`	Solo vuelve a sembrar (idempotente). Requiere install previo de V2
`trg_bank_seed_ve_drop.sql`	Quita trigger/funciones/catálogo. **No borra** `public.bank`

Paso a paso: cómo ejecutar

1. Primera vez (o pasar de V1 a V2)

```
```bash
psql -d <base> -f "/Users/kiberno/Documents/Requerimientos/TriggerBank/V2/trg_bank_seed_ve.sql"
```

Crea `bank\_catalog\_ve`, funciones de normalización/match, `trg\_bank\_seed\_ve` (`ALTER INSERT`), y siembra cada empresa activa de `public.enterprise`.

Si V1 ya estaba instalado, este script hace `CREATE OR REPLACE` y vuelve a sembrar. No hace falta drop previo.

### ### 2. Re-siembra (datos existentes, trigger ya instalado)

Útil si el install se corrió cuando no había empresas, o se agregó una empresa sin un INSERT en `bank`.

```
```bash
psql -d <base> -f "/Users/kiberno/Documents/Requerimientos/TriggerBank/V2/trg_bank_seed_ve_run.sql"
```

Equivale a:

```
```sql
SELECT public.fn_bank_seed_catalog_all_enterprises();
```

Si V1 está instalado **\*\*sin\*\*** haber corrido el install de V2, `fn\_bank\_seed\_catalog\_all\_enterprises` no existe: hay que correr primero `trg\_bank\_seed\_ve.sql`.

### ### 3. Una sola empresa

```
```sql
SELECT public.fn_bank_seed_catalog_for_enterprise(<id_enterprise>, '<co_enterprise>');
```

Devuelve cuántas filas insertó (0 si no faltaba nada).

4. Rollback de objetos (no de datos)

```
```bash
psql -d <base> -f "/Users/kiberno/Documents/Requerimientos/TriggerBank/V2/trg_bank_seed_ve_drop.sql"
```

Los bancos ya insertados en `public.bank` **\*\*permanecen\*\***.

### ## Verificación

1. Banco llamado "Venezuela" o "Banco de Venezuela": no debe aparecer un segundo BANCO DE VENEZUELA (0102).
2. Código `0102` con nombre distinto al CSV: el nombre **\*\*no cambia\*\***; el CSV no entra.
3. Empresa sin conflictos: se insertan los faltantes (hasta 25) con `co\_operation = 'I'` y `da\_update = now()`.
4. Segundo INSERT en la misma empresa o segundo `run.sql`: 0 insertados extra.
5. Empresa en `enterprise` sin filas en `bank`: el backfill de V2 igual siembra el catálogo.

## ## Notas técnicas

- Reentrada: ``denario.bank_seeding = 1`` y ``pg_trigger_depth()`` para que los INSERT del seed no disparen el trigger en bucle.
- ``id_bank``: secuencia de ``bank`` o ``MAX(id_bank)+1``.
- El trigger **\*\*no\*\*** se dispara al crear el trigger; por eso el install incluye `backfill`.
- No se copia a Flyway / Webservice en esta entrega.